NODE.JS BACKEND DEVELOPMENT

Phase 1 Summary

Days 1–4 • Beginner Foundations

In this guide: Node.js basics, built-in modules, CommonJS modules, npm, callbacks, Promises, and async/await.

A revision guide to help you refresh the foundations before continuing with HTTP and Express.

Day 1 — Introduction to Node.js

What is Node.js?
Node.js is a JavaScript runtime that lets you run JavaScript outside a web browser. It is commonly used to build backend applications and servers.

Run JavaScript with Node.js

```
node app.js
```

Print output

```
console.log("Hello Node.js");
```

Command-line arguments: `process.argv` contains command-line values. Index 0 is the Node executable path, index 1 is the script path, and index 2 onward contains values you provide. Arguments are strings, so convert numeric input with `Number()`.

```
const name = process.argv[2];
const age = Number(process.argv[3]);
console.log(name, age);
```

Built-in modules

Node.js includes built-in modules that do not need separate installation. Import them with `require()`.

```
const fs = require("fs");
const os = require("os");
```

Useful fs methods: `writeFileSync()` writes a file; `readFileSync()` reads it; `appendFileSync()` adds content; `unlinkSync()` deletes a file. Use `"utf8"` to read text as a string.

```
const fs = require("fs");
fs.writeFileSync("notes.txt", "Hello");
const data = fs.readFileSync("notes.txt", "utf8");
console.log(data);
```

Useful os methods: `os.platform()`, `os.arch()`, `os.freemem()`, and `os.totalmem()`.

Sync vs async: synchronous work blocks the current JavaScript execution until it finishes. Asynchronous work lets the program continue while waiting for an operation.

Day 2 — Modules in Node.js

Why use modules?
Modules let you split a program across files. This makes code more organized, reusable, and easier to maintain.

Export and import multiple functions

```
// calculator.js
function add(a, b) { return a + b; }
function subtract(a, b) { return a - b; }
module.exports = { add, subtract };

// app.js
const calculator = require("./calculator");
console.log(calculator.add(5, 3));
```

Import with destructuring

```
const { add, subtract } = require("./calculator");
console.log(add(5, 3));
```

Export a single function

```
// greeting.js
function greet(name) {
  return `Hello, ${name}`;
}
module.exports = greet;

// app.js
const greet = require("./greeting");
console.log(greet("Alex"));
```

CommonJS: uses `require()` to import and `module.exports` (or `exports`) to export. **ES Modules:** uses `import` and `export`. This course has focused on CommonJS.

Important: `exports` initially refers to `module.exports`, so `exports.greet = greet` works. Reassigning `exports` alone does not replace `module.exports`.

Day 3 — npm and Packages

What is npm?
npm is the package manager commonly used with Node.js. It helps initialize projects and install, remove, and manage third-party packages.

Create a project

```
npm init -y
```

This creates `package.json`, which stores project information, scripts, and dependencies.

Install and use a package

```
npm install validator
```

```
const validator = require("validator");
console.log(validator.isEmail("hello@example.com"));
console.log(validator.isURL("https://example.com"));
```

Project files:
`package.json` describes the project and lists dependencies. `package-lock.json` records the resolved dependency versions/tree. `node_modules` contains installed packages.

Remove a package

```
npm uninstall validator
```

npm scripts

```
"scripts": {
  "start": "node app.js",
  "check": "node app.js"
}

npm start
npm run check
```

Restore dependencies: if `node_modules` is missing, run `npm install` in the project folder. npm uses the package files to install dependencies again.

Remember: `npm start` is the usual shortcut for the start script. For other scripts, use `npm run scriptName`.

Day 4 — Asynchronous JavaScript

Why async?
Some operations take time, such as reading files, contacting an API, or waiting for a database. Asynchronous code lets the program continue instead of blocking while it waits.

Callbacks

A callback is a function passed to another function so it can be called later.

```
function processUser(name, callback) {
  callback(name);
}

processUser("Sam", (user) => {
  console.log("Welcome " + user);
});
```

Promises

A Promise represents the eventual result of an asynchronous operation. Its states are **pending**, **fulfilled**, and **rejected**. The Promise executor receives `resolve` and `reject` functions.

```
const checkAge = new Promise((resolve, reject) => {
  const age = 20;
  if (age >= 18) resolve("Allowed");
  else reject("Not allowed");
});

checkAge
  .then((result) => console.log(result))
  .catch((error) => console.log(error));
```

Promise chaining: return the next Promise from a `.then()` callback so the following `.then()` waits for it.

```
saveOrder()
  .then((order) => processPayment(order))
  .then((payment) => sendEmail(payment))
  .then(() => console.log("Order complete"))
  .catch((error) => console.log(error));
```

async / await

`async` makes a function return a Promise. `await` pauses that async function until the awaited Promise settles; it does not pause the entire program. Use `try/catch` to handle errors.

```
const fs = require("fs/promises");

async function readNotes() {
  try {
    const data = await fs.readFile("notes.txt", "utf8");
    console.log(data);
  } catch (error) {
    console.log("Could not read file:", error.message);
  }
}

readNotes();
```

Event-loop reminder: synchronous statements run first. When an async function reaches `await`, that function pauses and the rest of the program can continue.

Phase 1 — Quick Revision Checklist

Topic	Make sure you can…
Node.js	Explain what Node.js is and run a .js file with `node`.
CLI arguments	Read `process.argv` and convert strings when needed.
Built-in modules	Use basic `fs` and `os` methods.
Modules	Export functions with `module.exports` and import with `require`.
npm	Initialize a project, install/uninstall packages, and run scripts.
Async basics	Explain callbacks and why asynchronous work is useful.
Promises	Explain Promise states, `resolve`/`reject`, `then`/`catch`.
async/await	Use `await` inside an async function and handle errors with `try/catch`.

Common mistakes to avoid

- Forgetting that `process.argv` values are strings.
- Importing a local file without its relative path (for example, `./calculator`).
- Confusing `package.json` with `package-lock.json`.
- Forgetting to return a Promise inside `.then()` when the next step must wait.
- Thinking `await` blocks the entire application; it pauses only the current async function.

End of Phase 1 • Days 1–4